

Sudo con env_keep (LD_PRELOAD)

Objetivos de aprendizaje

Identificar la superficie, explicar la causa, diseñar una confirmación mínima, interpretar evidencia y proponer una mitigación causal.

Prerrequisitos

Flujo source-transformación-validación-sink, parsers, permisos y metodología de hipótesis. Revisa los prerrequisitos declarados en los metadatos.

Fundamentos técnicos

LD_PRELOAD fuerza al cargador a inyectar una librería tuya antes que las del sistema en cualquier programa. Si sudo está configurado con env_keep+=LD_PRELOAD, esa variable sobrevive al sudo y puedes precargar una .so maliciosa en un comando que tengas permitido ejecutar con sudo, corriendo tu código como root. Variante avanzada de sudo-abuse.

El admin conserva variables de entorno por comodidad sin darse cuenta de que LD_PRELOAD permite inyectar código. Tu .so define una función (por ejemplo el constructor) que lanza una shell; al cargarla en el proceso privilegiado, esa shell es root.

Modelo mental

Entrada controlada -> transformaciones -> validación -> componente final
-> sink -> efecto observable -> decisión

El paso crítico es demostrar qué componente consume el valor y con qué identidad o privilegios.

Superficie de ataque

Cuando sudo -l (sudo-abuse) muestra 'env_keep+=LD_PRELOAD' (o LD_LIBRARY_PATH) y tienes algún comando permitido con sudo, aunque sea inofensivo.

Cómo identificarla

- sudo -l muestra 'env_keep+=LD_PRELOAD' en la política.
- Tienes al menos un comando ejecutable con sudo.
- Puedes compilar en la máquina (gcc presente) o subir la .so ya compilada.

Preguntas que debo hacerme

- ¿Qué dato controlo exactamente y en qué formato viaja?
- ¿Qué transformaciones y normalizaciones ocurren antes del sink?
- ¿Qué identidad ejecuta la operación y qué permisos tiene?
- ¿Qué otra explicación produciría la misma señal?
- ¿Cuál es la prueba de menor riesgo que separa ambas explicaciones?

Prueba mínima

Ejecuta el comando permitido con sudo LD_PRELOAD=/tu.so y observa si tu código corre.

Evidencia esperada: Un prompt de root al ejecutar el comando con la variable precargada.

Construcción progresiva del payload

1. Reproduce una entrada válida.
2. Aísla un único valor controlable.

3. Define la primitiva mínima indicada por la fuente.
4. Predice resultado y control negativo.
5. Aplica una sola adaptación cuando haya evidencia de restricción.
6. Confirma de forma reproducible antes de ampliar impacto.

Capa 1: prueba documentada

```
gcc -fPIC -shared -o /tmp/pe.so pe.c -nostartfiles
```

Objetivo y contexto: Compila tu librería como shared object. -shared la hace precargable; -nostartfiles evita dependencias que romperían la carga en el proceso víctima.

Resultado esperado: El fichero /tmp/pe.so. Su código (una función _init/constructor que hace setuid(0); system("/bin/bash")) se ejecutará al precargarse.

Capa 2: prueba documentada

```
sudo LD_PRELOAD=/tmp/pe.so <comando_permitido>
```

Objetivo y contexto: Al ejecutar el comando permitido con sudo, LD_PRELOAD (conservado por env_keep) carga tu .so en el proceso root y dispara tu shell.

Resultado esperado: Un prompt de root (id = uid=0). Si da error de carga, revisa que compilaste con -nostartfiles y que env_keep realmente conserva la variable.

Anatomía de los payloads

La primera prueba es gcc -fPIC -shared -o /tmp/pe.so pe.c -nostartfiles. Separa sus delimitadores, operadores, opciones, operandos y variables; algunos elementos no estarán presentes según el contexto. Las comillas, barras y separadores pertenecen a una capa concreta. Usa la explicación de cada capa para determinar qué símbolo altera sintaxis y cuál transporta datos.

Variaciones según el contexto

No traslades una prueba entre sistemas operativos, motores, frameworks o versiones sin revisar sintaxis y comportamiento. Conserva la primitiva y vuelve a serializarla para el parser real.

Filtros y bypasses

No existe un bypass universal. Registra la forma enviada, la forma decodificada, la comparación, la normalización y la forma consumida. Una adaptación es válida solo si demuestra una discrepancia concreta; si la validación ocurre sobre la forma canónica y expresa la propiedad correcta, la variante no debe funcionar.

Evidencias de confirmación

Un prompt de root al ejecutar el comando con la variable precargada.

Usa además una entrada inocua y un control negativo. No confundas reflexión, error genérico o demora aislada con confirmación.

Escalado de impacto

- Confirma env_keep+=LD_PRELOAD en sudo -l.
- Escribe una .so mínima cuyo constructor lance /bin/bash y quite privilegios (setuid(0)).
- Compíllala como shared object.
- Ejecuta un comando permitido con sudo LD_PRELOAD=/ruta/tu.so; obtienes shell root.

Amplía únicamente dentro del laboratorio y cuando cada salto aporte una capacidad nueva demostrable.

Errores frecuentes

- sudo -l no muestra env_keep para LD_PRELOAD/LD_LIBRARY_PATH.

- No tienes gcc en la maquina ni forma de subir una .so ya compilada.
- Ejecutar la prueba sin adaptar variables ni versión.
- Cambiar varias capas a la vez.
- Omitir el control negativo o no guardar evidencia.

Diagnóstico de payloads fallidos

Síntoma	Posible causa	Prueba de diagnóstico	Adaptación
Rechazo inmediato	Formato o precondition	Repetir entrada válida	Corregir transporte
Sin diferencia	Entrada ignorada o canal ciego	Marcador y control negativo	Buscar evidencia adecuada
Error del componente	Contexto o versión	Reducir a prueba mínima	Consultar manual detectado
Resultado parcial	Permisos/restricción	Comprobar identidad y alcance	Reducir primitiva

Mitigaciones

Eliminar el dato controlable del sink cuando sea posible; usar APIs estructuradas, allowlists sobre valores canónicos, privilegio mínimo, autorización en servidor y registros que permitan detectar abuso. La defensa concreta debe impedir la causa explicada en Fundamentos técnicos.

Relación con pentesting y certificaciones

Se espera reconocer la señal, justificar la prueba elegida, adaptar variables, interpretar salida y documentar impacto y mitigación. La puntuación debe premiar razonamiento y evidencia, no memoria literal.

Caso guiado

Parte de una señal de la lista anterior. Escribe observación e hipótesis, ejecuta la prueba mínima, compara con el control y clasifica el resultado como no confirmado, indicio o confirmación. Solo entonces sigue los pasos de escalado relevantes.

Caso de adaptación

Si la prueba básica falla, no cambies caracteres al azar. Comprueba primero transporte, parser, versión, permisos y canal de evidencia. Diseña una segunda prueba que discrimine entre las dos causas más probables.

Ejercicios

1. Señala source, transformaciones y sink en el caso guiado.
2. Explica qué evidencia refutaría la hipótesis.
3. Descompón la primera prueba documentada por opciones y argumentos.
4. Propón un control negativo y una mitigación causal.

Resumen

Si sudo conserva LD_PRELOAD, cargas una librería tuya en un comando permitido y ejecutas código como root. La técnica queda confirmada solo cuando una prueba mínima produce la evidencia prevista y descarta explicaciones alternativas.

Chuleta operativa

1. Confirmar alcance y condiciones.
2. Identificar entrada, componente y permisos.

3. Ejecutar prueba mínima y control.
4. Interpretar señal antes de escalar.
5. Guardar evidencia y mitigación.

Referencias

- [Concepto fuente de THM Fieldbook \(sudo-ld-preload\)](#)
- [Referencia técnica externa](#)

Navegación

Anterior: [Construcción de payloads](#). Índice: [curso](#). Ejercicios y chuleta se enlazarán desde la matriz de trazabilidad.